Numerical Methods for Computer Science

Laurin Seeholzer

October 29, 2026

Contents

1 Introduction **3**
- 1.1 Why Numercial Methods in Computer Science . . . 3
  - 1.1.1 Approximation Strategies: Promises and Limitations . . . 3
  - 1.1.2 Core Topics in Numerical Methods . . . 3
  - 1.1.3 Key Design Goals in Numerical Methods . . . 4
- 1.2 Rounding Errors . . . 4
  - 1.2.1 Machine Numbers . . . 4
  - 1.2.2 Numerical Differentiation . . . 5
  - 1.2.3 Convergence Acceleration via Richardson Extrapolation . . . 6
- 1.3 Matrices & Compute-Work . . . 7

2 Interpolation **8**
- 2.1 Newton . . . 8
- 2.2 Lagrange . . . 9
- 2.3 Barycentric Interpolation-formula . . . 9
- 2.4 Chebyshev . . . 10
- 2.5 Trigonometric Interpolation . . . 11
- 2.6 Piecewise Polynomial Interpolation . . . 14
- 2.7 Piecewise Linear Interpolation . . . 15
- 2.8 Cubic Hermite Interpolation . . . 16
- 2.9 Spline Interpolation . . . 17

3 Quadrature **20**
- 3.1 Polynomial Interpolation . . . 20
- 3.2 Composite Quadrature Rules . . . 21
  - 3.2.1 The Midpoint Rule . . . 21
  - 3.2.2 The Trapezoidal Rule . . . 21
  - 3.2.3 Simpson's Rule . . . 22
  - 3.2.4 Smoothness . . . 22
  - 3.2.5 Romberg Extrapolation . . . 23
  - 3.2.6 High-Order Rules . . . 23
- 3.3 Adaptive Quadrature . . . 24
- 3.4 Higher-Dimensional Quadrature . . . 25
  - 3.4.1 Tensor Product Rules (Standard Approach) . . . 25
  - 3.4.2 Sparse Grids (Smolyak / Combination Technique) . . . 26
  - 3.4.3 Monte Carlo Integration . . . 27
  - 3.4.4 Quasi-Monte Carlo and Halton Sequences . . . 29

1 Introduction

1.1 Why Numercial Methods in Computer Science

Numerical Analysis (*Numerische Methoden*) focuses on developing efficient algorithms and accurate approximations to solve mathematical problems that cannot be solved analytically.

1.1.1 Approximation Strategies: Promises and Limitations

A central task in numerical methods is function approximation. Depending on the properties of the target function and the domain of interest, different representation bases are chosen:

- **Taylor Polynomial Approximation:** Local polynomial approximations rely on Taylor series expansions. While effective locally, Taylor polynomials require high-order derivatives and strict smoothness assumptions ($f \in C^{m+1}$). Furthermore, convergence is strictly local around an expansion point x_0.
- **Global Polynomial Spaces & Projection:** To obtain global approximations without requiring derivatives, one can select a finite-dimensional subspace (e.g., polynomials up to degree m) spanned by a chosen basis, such as monomials $\{1, x, x^2, \ldots, x^m\}$ or orthogonal polynomials. The best approximation is then found by projecting the function onto this subspace under a chosen norm (e.g., L^2-norm).
- **Fourier Approximation:** For periodic functions, Fourier series provide a global approximation by expanding $f \in L^2(0,1)$ in terms of complex exponentials:

$$f(t) = \sum_{k=-\infty}^{\infty} \hat{f}(k) e^{2\pi i k t}, \quad \text{where } \hat{f}(k) = \int_0^1 f(t) e^{-2\pi i k t} dt$$

  Fourier approximations achieve rapid (exponential) convergence for smooth periodic functions without requiring explicit derivative calculations.

1.1.2 Core Topics in Numerical Methods

Numerical analysis encompasses several fundamental domains aimed at constructing scalable and stable algorithms:

- **Interpolation:** Constructing an approximating function that passes exactly through a given set of discrete data points (x_j, y_j).
- **Numerical Quadrature:** Efficiently evaluating definite integrals $\int_a^b f(t)dt$ using weighted sums at discrete node points.
- **Numerical Linear Algebra:** Solving large linear systems $A\underline{x} = \underline{b}$ efficiently and stably.
- **Linear Least Squares (*Lineare Ausgleichsrechnung*):** Finding optimal parameters by minimizing $\|A\underline{x} - \underline{b}\|_2$ for overdetermined systems.
- **Nonlinear Systems of Equations & Least Squares:** Finding roots $\underline{f}(\underline{x}^*) = \underline{0}$ or solving non-linear optimization problems $\min_{\underline{p}} \|f(\underline{p}, \cdot) - y\|_2$.

1.1.3 Key Design Goals in Numerical Methods

When designing numerical algorithms, the main objectives are:

1. **Efficiency & Time Complexity:** Minimizing computation time and floating-point operations.
2. **Memory Footprint:** Optimizing memory usage for large-scale calculations.
3. **Quantifiable Accuracy & Convergence:** Guaranteeing and bounding errors for convergent sequences.
4. **Generality & Stability:** Ensuring robustness against rounding errors and subtractive cancellation.

1.2 Rounding Errors

Definition 1.1. Absolute Error: $\|\tilde{x} - x\|$ with $\|\cdot\|$ a norm in $\mathbb{R}^n$.

Definition 1.2. Relative Error: $\frac{\|\tilde{x}-x\|}{\|\tilde{x}\|}$ for $\tilde{x} \neq 0$, with $\|\cdot\|$ a norm in $\mathbb{R}^n$.

1.2.1 Machine Numbers

Remark 1.3. In modern computer systems, numbers are represented as floating-point numbers using a mantissa-exponent representation.

Definition 1.4. The **Mantissa-Exponent** representation of a number is $\pm M \cdot B^E$, where M is the mantissa containing the significant digits and precision, $B \geq 2$ is the base (typically 2 or 10), and $E \in \{e_{\min}, \ldots, e_{\max}\}$ is the exponent determining the scale or magnitude of the number by shifting the radix point.

Example 1.5. Cancellation in Quadratic Equations: The polynomial $p(x) = (x - c)(x - 1/c) = x^2 - (c + 1/c)x + 1$ has roots c and $1/c$. When using the standard quadratic formula (*Mitternachtsformel*) to calculate these roots: The relative error for the smaller root $1/c$ is significantly larger than for the larger root c. This occurs because calculating the smaller root requires subtracting two nearly equal values ($-a$ and $\sqrt{D}$), leading to a loss of significant digits.

Code Example 1.6

```
D = a**2 - 4*b
wD = sqrt(D)
z = ((-a + wD)/2, (-a - wD)/2)
```

Definition 1.7. This phenomenon is called **cancellation** (*Auslöschung*). It describes how the subtraction of two nearly equal numbers dramatically amplifies relative errors, even though the error of the subtraction operation itself is negligible.

Remark 1.8. Stable Alternative via Vieta's Formulas: To avoid cancellation when solving $x^2 + ax + b = 0$, compute the numerically stable larger root x_2 first, and obtain the smaller root x_1 using $x_1 x_2 = b \implies x_1 = b/x_2$:

Code Example 1.9

```
x2 = (sqrt(D) - a)/2 if a < 0 else (-sqrt(D) - a)/2
x1 = b / x2
```

1.2.2 Numerical Differentiation

Remark 1.10. We can use the forward difference quotient to approximate local derivatives:

$$f'(x) \approx \frac{f(x+h) - f(x)}{h}$$

Code Example 1.11

```
f = exp; df = exp; x = 0; h = 0.1
for k in range(1, 21):
    ndf = (f(x+h) - f(x)) / h
    h *= 0.1
```

$\log_{10}(h)$	relativer Fehler
-1	0.05170918075648
-2	0.005016708416 79
-3	0.000500166708 38
-4	0.000050001667 14
-5	0.000005000006 96
-6	0.000000499962 18
-7	0.000000049433 68
-8	0.000000006077 47
-9	0.000000008274 037
-10	0.000000008274 037
-11	0.000000008274 037
-12	0.000088900582 34
-13	0.000799277837 36
-14	0.000799277837 36
-15	0.110223024625 16
-16	1.000000000000 00

Remark 1.12. Observe that the approximation error initially decreases as h grows smaller, reaching optimal accuracy around $h \approx \sqrt{2 \cdot \text{eps}} \approx 10^{-8}$. However, as $h \to 0$ further, cancellation errors dominate because $f(x+h) \approx f(x)$, causing total accuracy to rapidly degrade.

Theorem 1.13. Complex Step Derivative: If f is analytic, we can use a complex step to perform numerically stable differentiation without cancellation. Using taylor approximation we get:

$$f(x_0 + ih) = f(x_0) + ihf'(x_0) - \frac{h^2}{2}f''(x_0) - \frac{ih^3}{6}f'''(x_0) + \mathcal{O}(h^4)$$

Taking the imaginary part yields:

$$\text{Im}\left(f(x_0 + ih)\right) = hf'(x_0) - \frac{h^3}{6}f'''(x_0) + \mathcal{O}(h^5)$$

hence by rearanging we get

$$\implies f'(x_0) = \frac{\text{Im}\left(f(x_0 + ih)\right)}{h} + \mathcal{O}(h^2) \approx \frac{\text{Im}\left(f(x_0 + ih)\right)}{h}$$

Since this formulation involves no subtraction of function values, it is immune to subtractive cancellation for arbitrarily small values of h.

1.2.3 Convergence Acceleration via Richardson Extrapolation

Theorem 1.14. Symmetric Difference Quotient and Error Expansion: Using the central difference quotient $d(h) = \frac{f(x+h)-f(x-h)}{2h}$, Taylor expansion yields an error expansion containing only even powers of h:

$$f'(x) - d(h) = c_1h^2 + c_2h^4 + c_3h^6 + \cdots + \mathcal{O}(h^{2n+2})$$

Theorem 1.15. Richardson Extrapolation: By combining approximations evaluated at step sizes h and $h/2$, the leading error term c_1h^2 can be eliminated:

$$f'(x) = d(h) + c_1h^2..., \quad f'(x) = d(\frac{h}{2}) + c_1\frac{h}{2}^2... \quad \implies \quad d_1(h) = \frac{4d(h/2) - d(h)}{3}$$

The refined approximation $d_1(h)$ achieves an increased convergence order of $\mathcal{O}(h^4)$:

$$f'(x) - d_1(h) = \tilde{c}_2h^4 + \tilde{c}_3h^6 + \dots$$

Definition 1.16. Richardson Scheme: Higher-order accuracy can be computed recursively using the triangular scheme ($R_{l,k}$ table) for levels $l = 0, 1, 2, \dots$ and

extrapolation steps $k = 1, 2, \ldots$:

$$R_{l,0} = d\left(\frac{h}{2^l}\right), \quad R_{l,k} = \frac{4^k R_{l,k-1} - R_{l-1,k-1}}{4^k - 1}$$

Each column k increases the order of convergence such that:

$$f'(x) - R_{l,k} = \mathcal{O}\left(\left(\frac{h}{2^l}\right)^{2k+2}\right)$$

This allows reaching high numerical precision at significantly larger step sizes h, effectively suppressing rounding and cancellation errors.

1.3 Matrices & Compute-Work

Important 1.17. In NumCS we define the Compute-Work as the theoretical, asymototic number of elementary operations. Observe that this definition has (almost) noting to do with the actual compute-time on a chip.

Example 1.18. Worst case Compute-Work for Gauss-Elimination

- General: $O(n^3)$
- Upper Triangular: $O(n^2)$
- Diagonal: $O(n)$

Important 1.19. Numpy does not automatically adjust algorithms to the given inputs. We have to manually exploit the structure of ex. matrices and vectors.

Example 1.20. Let D be a diagonal matrix and let A be some other matrix. Using $np.dot(D, A)$ will result in $O(n^3)$ runtime just like any normal matrix multiplication. We have to manually exploit the structure of D by broadcasting the diagonal entries to a vector and then use numpy to multiply this vector by each row.

Example 1.21. Let $R = triu(AB)$ be the uppertringular matrix of the matrix product of $A, B \in R^{p,n}$ calculating $y = Rx$ takes $O(pn^2)$ using a standard numpy implementation

$$y = np.triu(A@B.T)@x$$

Let $p = 1$ we can represent rewrite this to $A'@T@B'@x$ where A', B' are diagonal matrices made from the rank one matrices A, B, T is a uppertringular matrix filled with ones. Observe that these products can are just (reverse) partialsums we can calculate using cumulative sum in numpy in $O(n)$. We can use this concept for any p and get a runtime of $O(pn)$.

2 Interpolation

Definition 2.1. Interpolation: Given exact values

$$y_1 = f(x_1, x_2 \dots), y_2 = f(x_i, x_{i+1} \dots) \dots$$

of an unknonw function f determine an interplant $\tilde{f}$ such that

$$\tilde{f}(x_1, x_2, \dots) = y_1$$

Remark 2.2. We assume f to have specific properties (smooth, monotone, etc.) $f \in V$ and choose a Subspace $\tilde{f} \in V_n$ together wiht a basis $b \in V_n$. Different spaces and basis are possible and used for different applications.

Example 2.3. If we choose V_n as the set of all polynomials of degree $\leq n$ we are calculating a polynomial interpolation of f.

Example 2.4. Using $n+1$ datapoints we can solve the linear system of equations of the $n+1$ datapoints to get a polynomial of degree n.

$$\begin{aligned} a_n x_1^n + a_{n-1} x_1^{n-1} \dots a_0 &= y_1 \\ a_n x_2^n + a_{n-1} x_2^{n-1} \dots a_0 &= y_2 \\ &\dots \\ a_n x_{n+1}^n + a_{n-1} x_{n+1}^{n-1} \dots a_0 &= y_{n+1} \end{aligned}$$

Solving this takes $O(n^3)$ and might come with unreasonably large rounding error.

Definition 2.5. The matrix A formed by this system of linear equations is called a **Vandermonde matrix**.

2.1 Newton

Definition 2.6. Newton-Basis: The newton basis $N_n(x)$ is

$$\prod_{i=0}^{n-1} (x - x_i) = (x - x_1)(x - x_2) \dots$$

Theorem 2.7. Using the newton basis we can continously increase the degree of interpolation without having to solve the whole system of equations at once.

$$p(x_0) = y_0 \implies p_0(x) = y_0$$

$$p(x_0, x_1) = p_0, y_1 \implies p_1(x) = p_0(x) + \frac{y_1 - y_0}{x_1 - x_0}(x - x_0)$$

$$p(x_0, x_1, x_2) = p_1, p_1, y_2 \implies p_2(x) = p_1(x) + \frac{\frac{y_2-y_1}{x_2-x_1} - \frac{y_1-y_0}{x_1-x_0}}{x_2 - x_0}(x - x_0)(x - x_1)$$
$$\dots$$

This can be interpreted as

$$p_i(x) = b_1 + b_2 N_1 + b_3 N_2 + \dots$$

where b_i is the i-th devided difference.

Definition 2.8. The **i-th devided difference** b_i is definied by the follwing recurrence relation

$$b_i = f[x_0, \dots x_i] = \frac{f[x_1, \dots x_i] - f[x_0, \dots x_{i-1}]}{x_i - x_0}$$

where $f[x_i] = y_i$

2.2 Lagrange

Definition 2.9. Lagrange-polynomials are polynomials of the form

$$l_i(x) = \prod_{j=0, j\neq i}^{n} \frac{x - x_j}{x_i - x_j} = \frac{(x - x_1)(x - x_2)(x - x_3)\dots}{(x_i - x_1)(x_i - x_2\dots)}$$

Theorem 2.10. Observe that for such a largrange polynomial $l_i(x)$ we have

$$l_i(x) = 0 \forall x_j \text{with} j \neq i \quad \text{and} \quad l_i(x_i) = 1$$

Theorem 2.11. Because the above holds, given a set of datapoints $(x_1, y_1) \dots (x_n, y_n)$ we can construct a polynomial that fullfilles all of the n conditions

$$p_n(x) = \sum_{i=0}^{n} y_i l_i(x)$$

2.3 Barycentric Interpolation-formula

Definition 2.12. We define the **barycentric weights** w_i and the node polynomial

$\ell(x)$ for a set of nodes $x_0, \ldots, x_n$ as

$$w_i = \prod_{j=0, j\neq i}^{n} \frac{1}{x_i - x_j} \qquad \ell(x) = \prod_{i=0}^{n} (x - x_i)$$

Theorem 2.13. Using these definitions, we can rewrite the Lagrange basis polynomials as $l_i(x) = \ell(x)\frac{w_i}{x-x_i}$. Substituting this into the standard Lagrange polynomial yields the **barycentric interpolation formula**:

$$p_n(x) = \ell(x) \sum_{i=0}^{n} \frac{w_i}{x - x_i} y_i$$

Theorem 2.14. Derivation of the second form: Because an interpolating polynomial reproduces constant functions exactly, interpolating the function $f(x) = 1$ (where $y_i = 1 \forall i$) gives:

$$1 = \ell(x) \sum_{i=0}^{n} \frac{w_i}{x - x_i}$$

Dividing the first form of the barycentric formula by this identity eliminates the node polynomial $\ell(x)$ and yields the **second (or true) barycentric interpolation formula**:

$$p_n(x) = \frac{\sum_{i=0}^{n} \frac{w_i}{x-x_i} y_i}{\sum_{i=0}^{n} \frac{w_i}{x-x_i}}$$

Remark 2.15. The second form is numerically highly stable. Once the weights w_i are precomputed (which takes $O(n^2)$), evaluating $p_n(x)$ at any new point x takes only $O(n)$ operations. Furthermore, adding a new data point only requires updating the weights, which is much cheaper than recalculating the entire Lagrange polynomial.

2.4 Chebyshev

Definition 2.16. Using equidistant nodes for polynomial interpolation of high degree often leads to severe oscillations at the edges of the interval. This is known as **Runge's phenomenon**. To minimize this error, we must choose our interpolation nodes strategically.

Theorem 2.17. The **error of polynomial interpolation** is given by:

$$f(x) - p_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \prod_{i=0}^{n} (x - x_i)$$

for some ξ in the interpolation interval. Since we cannot control the $(n+1)$-th derivative of the unknown function f, we must minimize the maximum absolute value of the node polynomial $\prod_{i=0}^{n}(x - x_i)$.

Definition 2.18. The roots of the Chebyshev polynomials of the first kind are called **Chebyshev nodes**. On the standard interval $[-1, 1]$, the $n+1$ nodes are defined as:

$$x_i = \cos\left(\frac{2i+1}{2n+2}\pi\right) \quad \text{for } i = 0, 1, \ldots, n$$

Remark 2.19. If the interpolation interval is $[a, b]$ instead of $[-1, 1]$, the nodes can be shifted and scaled using an affine transformation:

$$x_i^{[a,b]} = \frac{a+b}{2} + \frac{b-a}{2}x_i$$

Theorem 2.20. Using Chebyshev nodes minimizes the maximum norm of the node polynomial on $[-1, 1]$:

$$\max_{x\in[-1,1]} \left|\prod_{i=0}^{n}(x - x_i)\right| = \frac{1}{2^n}$$

This node placement guarantees uniform convergence for sufficiently smooth functions and effectively suppresses Runge's phenomenon by clustering the nodes closer together at the edges of the interval.

2.5 Trigonometric Interpolation

Definition 2.21. Hilbert Space $L^2(0,1)$: The space of square-integrable complex-valued functions on the interval $[0, 1]$ is defined as:

$$L^2(0,1) := \left\{ f : (0,1) \to \mathbb{C} \;\middle|\; \|f\|_{L^2(0,1)} = \sqrt{\langle f, f\rangle_{L^2(0,1)}} < \infty \right\}$$

equipped with the inner product $\langle g, f\rangle_{L^2(0,1)} := \int_0^1 \overline{g(t)} f(t)\, dt$.

Remark 2.22. L^2-Convergence: A function f is the L^2-limit of a sequence of functions $f_n \in L^2(0,1)$ if:

$$\lim_{n\to\infty} \|f - f_n\|_{L^2(0,1)} = 0 \iff \lim_{n\to\infty} \int_0^1 |f(t) - f_n(t)|^2\, dt = 0$$

Unlike pointwise convergence, L^2-convergence implies that the total energy (inte-

grated squared difference) between f and f_n vanishes, allowing isolated point discrepancies.

Theorem 2.23. The complex exponential functions $\phi_k(x) = e^{2\pi ikx}$ for $k \in \mathbb{Z}$ form a complete orthonormal basis in $L^2(0,1)$:

$$\langle \phi_k, \phi_j \rangle_{L^2(0,1)} = \int_0^1 e^{-2\pi ikx} e^{2\pi ijx}\, dx = \begin{cases} 1, & \text{if } k = j \\ 0, & \text{if } k \neq j \end{cases}$$

Definition 2.24. Fourier Series: Any function $f \in L^2(0,1)$ can be represented by its Fourier coefficients $\hat{f}(k)$ as:

$$f(t) = \sum_{k=-\infty}^{\infty} \hat{f}(k) e^{2\pi ikt}$$

where the Fourier coefficients $\hat{f}(k)$ are given by the orthogonal projection onto the basis functions:

$$\hat{f}(k) = \int_0^1 f(t) e^{-2\pi ikt}\, dt$$

Remark 2.25. Intuition: Analogous to decomposing a 3D vector into orthogonal coordinate axes, $f(t)$ is decomposed into pure periodic waves $e^{2\pi ikt}$ of integer frequency k. The magnitude $|\hat{f}(k)|$ represents the amplitude of frequency k, while its angle represents phase shift. Parseval's identity guarantees energy conservation:

$$\sum_{k=-\infty}^{\infty} |\hat{f}(k)|^2 = \|f\|^2_{L^2(0,1)}$$

Remark 2.26. The **decay rate** of Fourier coefficients depends directly on the smoothness of f. If $f^{(n)}$ is integrable then: Smoother functions yield faster coefficient decay, leading to rapid convergence of the Fourier series.

Remark 2.27. The Fourier coefficients of the n-th **derivative** $f^{(n)}$ satisfy:

$$\widehat{f^{(n)}}(k) = (2\pi ik)^n \hat{f}(k)$$

Remark 2.28. Gibbs Phenomenon: At jump discontinuities, the truncated Fourier series overshoots near the edge. Increasing the number of terms shifts the overshoot closer to the discontinuity but does not eliminate its amplitude.

Definition 2.29. N-th Root of Unity: The primitive N-th root of unity is defined as:

$$\omega_N := \exp\left(\frac{-2\pi i}{N}\right) = \cos\left(\frac{2\pi}{N}\right) - i\sin\left(\frac{2\pi}{N}\right)$$

The roots of unity satisfy the discrete orthogonality relation:

$$\sum_{k=0}^{N-1} \omega_N^{kj} = \begin{cases} N, & \text{if } j \equiv 0 \pmod{N} \\ 0, & \text{if } j \not\equiv 0 \pmod{N} \end{cases}$$

Definition 2.30. Discrete Fourier Transform (DFT): Given N equidistant nodes $x_l = \frac{l}{N}$ for $l = 0, \ldots, N-1$, approximating the continuous Fourier coefficients $\hat{f}(k)$ via the trapezoidal rule yields the DFT:

$$\hat{f}(k) \approx \hat{f}_N(k) = \frac{1}{N} \sum_{l=0}^{N-1} f\left(\frac{l}{N}\right) \omega_N^{kl}$$

The transformation mapping samples $y = (f(x_0), \ldots, f(x_{N-1}))^T \mapsto c = F_N y$ is performed via the Fourier matrix $F_N = (\omega_N^{ij})_{i,j=0}^{N-1}$.

$$F_N = \begin{pmatrix} 1 & 1 & 1 & \cdots & 1 \\ 1 & \omega_N^1 & \omega_N^2 & \cdots & \omega_N^{N-1} \\ 1 & \omega_N^2 & \omega_N^4 & \cdots & \omega_N^{2(N-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega_N^{N-1} & \omega_N^{2(N-1)} & \cdots & \omega_N^{(N-1)(N-1)} \end{pmatrix} \implies \hat{f}(k) \approx F_N \cdot y$$

Remark 2.31. Fast Fourier Transform (FFT): Using a divide-and-conquer strategy (e.g., Cooley-Tukey algorithm), the matrix-vector multiplication $F_N y$ can be evaluated in $\mathcal{O}(N \log N)$ operations instead of naive $\mathcal{O}(N^2)$.

Remark 2.32. Aliasing: Sampling f at discrete nodes $x_l = \frac{l}{N}$ makes frequencies differing by integer multiples of N indistinguishable, since $\omega_N^{(k+mN)l} = \omega_N^{kl}$. To avoid aliasing high frequencies into lower modes, N must satisfy the Nyquist-Shannon sampling threshold relative to the highest frequency present in f.

Definition 2.33. Trigonometric Interpolating Polynomial: For N samples $y_l = f(t_l)$ at $t_l = \frac{l}{N}$, the trigonometric polynomial $p_N(t)$ satisfying $p_N(t_l) = f(t_l)$ is expressed in symmetric frequency order:

$$p_N(t) = \sum_{k=-N/2}^{N/2-1} \hat{f}(k) e^{2\pi i k t}$$

Remark 2.34. Standard FFT outputs store coefficients indexed in non-symmetric order $k \in \{0, \ldots, N-1\}$. Applying `fftshift` rearranges indices from $[0, N-1]$ to symmetric frequency spectrum $[-\frac{N}{2}, \frac{N}{2} - 1]$.

Example 2.35. Cyclic Convolution and DFT: The cyclic convolution $c = a \otimes b$ of two N-periodic vectors $a, b \in \mathbb{C}^N$ is defined as:

$$c_k = \sum_{n=0}^{N-1} a_n b_{(k-n) \pmod N}$$

the DFT converts cyclic convolution into pointwise vector multiplication:

$$\mathcal{F}_N(a \otimes b) = \mathcal{F}_N(a) \odot \mathcal{F}_N(b)$$

This allows evaluating cyclic convolution in $\mathcal{O}(N \log N)$ operations via:

$$a \otimes b = \mathrm{IFFT}(\mathrm{FFT}(a) \cdot \mathrm{FFT}(b))$$

Linear convolution of length-N sequences can be evaluated using cyclic convolution by zero-padding both sequences to length at least $2N - 1$.

Theorem 2.36. Zero-Padding for High-Resolution Evaluation: To evaluate an interpolating trigonometric polynomial $p_N(t)$ at $M \gg N$ fine grid points:

1. Compute $c = \mathrm{FFT}(y)$ of size N and center coefficients using `fftshift`.
2. Pad the frequency vector with $M - N$ zeros at the high-frequency boundaries.
3. Apply `ifftshift` and compute $\tilde{y} = \frac{M}{N}\mathrm{IFFT}(\tilde{c})$ of size M.

This yields exact evaluation of $p_N(t)$ on a fine grid without computing individual exponential sums.

2.6 Piecewise Polynomial Interpolation

Remark 2.37. Motivation & The Problem with High Degrees: As seen with Runge's phenomenon, fitting a single global polynomial of high degree n over an entire interval $[a, b]$ often leads to severe, explosive oscillations near the boundary points. Moreover, high-degree polynomial interpolation is non-local: perturbing a

single data point alters the interpolant everywhere across the entire domain.

Definition 2.38. Grid Partition & Piecewise Polynomial: To overcome this instability, we replace a single global polynomial with localized low-degree polynomials. We divide the domain $[a, b]$ into N subintervals using breakpoints (nodes):

$$\mathcal{G} := \{a = x_0 < x_1 < x_2 < \cdots < x_N = b\}$$

with subinterval lengths $h_j := x_j - x_{j-1}$ for $j = 1, \ldots, N$, and maximal mesh width $h := \max_{1 \le j \le N} h_j$.

A function $s : [a, b] \to \mathbb{R}$ is called a **piecewise polynomial** of degree d if on each subinterval $I_j = [x_{j-1}, x_j]$, s is given by a polynomial of degree at most d:

$$s|_{[x_{j-1}, x_j]} \in \mathcal{P}_d \quad \text{for each } j = 1, \ldots, N$$

Remark 2.39. Intuition of h-Refinement vs. p-Refinement: Instead of achieving higher accuracy by driving the polynomial degree $d \to \infty$ (p-refinement), piecewise interpolation keeps the degree d low (typically $d \in \{1, 3\}$) and increases accuracy by making the subintervals smaller, i.e., letting the mesh size $h \to 0$ (h-refinement). This completely avoids Runge's phenomenon and ensures that errors and local perturbations remain confined to a small neighborhood.

2.7 Piecewise Linear Interpolation

Definition 2.40. Piecewise Linear Interpolant: The simplest piecewise polynomial sets $d = 1$. Given data points $(x_0, y_0), \ldots, (x_N, y_N)$, we connect consecutive points (x_{j-1}, y_{j-1}) and (x_j, y_j) with straight line segments:

$$s(x) = y_{j-1} \frac{x_j - x}{h_j} + y_j \frac{x - x_{j-1}}{h_j} \quad \text{for } x \in [x_{j-1}, x_j]$$

This produces a globally continuous function $s \in C^0[a, b]$.

Remark 2.41. While $s(x)$ is continuous across the entire domain, its derivative jumps at the interior nodes x_j:

$$\lim_{x \nearrow x_j} s'(x) = \frac{y_j - y_{j-1}}{h_j} \neq \frac{y_{j+1} - y_j}{h_{j+1}} = \lim_{x \searrow x_j} s'(x)$$

Because the slopes from the left and right generally differ, the interpolant has sharp "corners" or "kinks" at the nodes, meaning $s \notin C^1[a, b]$.

Definition 2.42. Nodal Hat Basis Functions: Piecewise linear interpolation can also be expressed globally using the nodal basis (also called **tent** or **hat functions**) $B_j(x)$:

$$s(x) = \sum_{j=0}^{N} y_j B_j(x)$$

where each basis function $B_j(x)$ satisfies $B_j(x_k) = \delta_{jk}$ and is defined as:

$$B_j(x) = \begin{cases} \frac{x - x_{j-1}}{x_j - x_{j-1}}, & \text{if } x \in [x_{j-1}, x_j], \\ \frac{x_{j+1} - x}{x_{j+1} - x_j}, & \text{if } x \in [x_j, x_{j+1}], \\ 0, & \text{otherwise} \end{cases}$$

Theorem 2.43. Error Estimate for Linear Interpolation: For a twice continuously differentiable function $f \in C^2[a, b]$, the interpolation error on a mesh of width $h = \max_j h_j$ satisfies:

$$\|f - s\|_{L^\infty} \leq \frac{1}{8} h^2 \|f''\|_{L^\infty}$$

and for the derivative (away from the nodes):

$$\|f' - s'\|_{L^\infty} \leq \frac{1}{2} h \|f''\|_{L^\infty}$$

The error in function values converges quadratically with order $\mathcal{O}(h^2)$: halving the grid width cuts the error by a factor of 4.

2.8 Cubic Hermite Interpolation

Definition 2.44. Cubic Hermite Interpolation: Given both function values $y_j = f(x_j)$ and derivative values $c_j = f'(x_j)$ at the nodes $x_0, \ldots, x_N$, Cubic Hermite interpolation constructs a local degree-3 polynomial $s_j \in \mathcal{P}_3$ on each interval $[x_{j-1}, x_j]$ satisfying 4 conditions:

$$s_j(x_{j-1}) = y_{j-1}, \quad s_j(x_j) = y_j, \quad s_j'(x_{j-1}) = c_{j-1}, \quad s_j'(x_j) = c_j$$

Since a cubic polynomial $ax^3 + bx^2 + cx + d$ has 4 degrees of freedom, these 4 conditions determine $s_j(x)$ uniquely on each interval.

Remark 2.45. Continuity (C^1 Smoothness): Because adjacent polynomials on $[x_{j-1}, x_j]$ and $[x_j, x_{j+1}]$ match both the function value y_j and the slope c_j at their shared boundary point x_j, the global interpolant $s(x)$ has no sharp corners and is

continuously differentiable on all of $[a, b]$:

$$s \in C^1[a, b]$$

Definition 2.46. Hermite Cardinal Basis: Using the local normalized coordinate $t = \frac{x - x_{j-1}}{h_j} \in [0, 1]$, the cubic polynomial on $[x_{j-1}, x_j]$ can be written directly without solving any linear system:

$$s(x) = y_{j-1}\phi_0(t) + y_j\phi_1(t) + h_j c_{j-1}\psi_0(t) + h_j c_j \psi_1(t)$$

where the Hermite basis polynomials on $[0, 1]$ are given by:

$$\phi_0(t) = 1 - 3t^2 + 2t^3, \quad \phi_1(t) = 3t^2 - 2t^3$$

$$\psi_0(t) = t - 2t^2 + t^3, \quad \psi_1(t) = -t^2 + t^3$$

These basis functions satisfy cardinal properties such as $\phi_0(0) = 1, \phi_0(1) = 0, \phi_0'(0) = 0, \phi_0'(1) = 0$, isolating the influence of each node value and derivative.

Remark 2.47. Limitation in Practice: Cubic Hermite interpolation requires exact derivative values $c_j = f'(x_j)$ at all node points. In many practical applications, however, only the function values y_j are available. This motivates the need for **splines**, which achieve high smoothness without requiring prescribed derivatives.

2.9 Spline Interpolation

Definition 2.48. Spline Spaces ($\mathcal{S}_{d,\mathcal{G}}$): Let $\mathcal{G} = \{a = x_0 < x_1 < \cdots < x_N = b\}$ be a grid on $[a, b]$. The spline space of degree $d \geq 1$, denoted by $\mathcal{S}_{d,\mathcal{G}}$, is the set of functions that:

1. Are $(d-1)$-times continuously differentiable on $[a, b]$: $s \in C^{d-1}[a, b]$.
2. Consist of polynomials of degree at most d on each subinterval: $s|_{[x_{j-1}, x_j]} \in \mathcal{P}_d$ for $j = 1, \ldots, N$.

Formally:

$$\mathcal{S}_{d,\mathcal{G}} := \left\{ s \in C^{d-1}[a, b] \;\middle|\; s|_{[x_{j-1}, x_j]} \in \mathcal{P}_d,\ j = 1, \ldots, N \right\}$$

The dimension of the spline space $\mathcal{S}_{d,\mathcal{G}}$ is:

$$\dim \mathcal{S}_{d,\mathcal{G}} = N + d$$

Remark 2.49. Dimension $N + d$: On N subintervals, each polynomial of degree d has $d + 1$ coefficients, giving a total of $N(d + 1)$ parameters. At each of the $N - 1$ internal nodes $x_1, \ldots, x_{N-1}$, the function and its first $d - 1$ derivatives must match

from the left and right. This imposes d continuity conditions per internal node, giving $d(N-1)$ linear constraints. Subtracting constraints from parameters:

$$\dim \mathcal{S}_{d,\mathcal{G}} = N(d+1) - d(N-1) = Nd + N - Nd + d = N + d$$

Therefore, to uniquely determine a cubic spline, we need **exactly 2 additional boundary conditions**.

Definition 2.50. Cubic Spline Boundary Conditions: To close the system and uniquely fix the 2 remaining degrees of freedom, one of the following boundary conditions is typically chosen:

1. **Clamped / Complete Splines (*Eingespannte Splines*):** Specifies exact boundary first derivatives:

   $$s'(x_0) = c_0 \quad \text{and} \quad s'(x_N) = c_N$$

   This is used when the physical boundary slopes (such as entry/exit velocities) are known.

2. **Natural Splines (*Natürliche Splines*):** Sets the second derivatives at the boundaries to zero:

   $$s''(x_0) = 0 \quad \text{and} \quad s''(x_N) = 0$$

   This means the curvature vanishes at the endpoints, so the spline transitions into a straight line at the boundary.

3. **Periodic Splines (*Periodische Splines*):** For periodic data where $y_0 = y_N$, enforces equal first and second derivatives at the endpoints:

   $$s'(x_0) = s'(x_N) \quad \text{and} \quad s''(x_0) = s''(x_N)$$

   This ensures that closed curves or periodic signals remain C^2-smooth across the period boundary.

4. **Not-a-Knot Condition:** Requires the third derivative $s'''(x)$ to be continuous across the first and last internal nodes:

   $$\lim_{x \nearrow x_1} s'''(x) = \lim_{x \searrow x_1} s'''(x) \quad \text{and} \quad \lim_{x \nearrow x_{N-1}} s'''(x) = \lim_{x \searrow x_{N-1}} s'''(x)$$

   Since two cubic polynomials that match up to the 3rd derivative must be identical, the first two intervals $[x_0, x_2]$ share the same single cubic polynomial, and the last two intervals $[x_{N-2}, x_N]$ share the same cubic polynomial. The nodes x_1 and x_{N-1} are effectively "not knots".

Theorem 2.51. Minimal Curvature Property of Natural Splines: Let s be the natural cubic spline interpolating the points $(x_j, y_j)_{j=0}^N$. Then s minimizes the total bending energy (graph curvature) among all twice continuously differentiable

interpolants:

$$\int_a^b |s''(x)|^2 \, dx \leq \int_a^b |g''(x)|^2 \, dx$$

for any function $g \in C^2[a, b]$ satisfying $g(x_j) = y_j$ for all $j = 0, \ldots, N$.

Remark 2.52. Computation via Tridiagonal Systems: Computing a cubic spline does not require solving an intractable dense matrix. The conditions of C^2 continuity lead to a strictly diagonally dominant **tridiagonal system of linear equations** for the derivative values $c_j = s'(x_j)$ (or second derivatives $s''(x_j)$). This tridiagonal system is always invertible and can be solved in only $\mathcal{O}(N)$ operations using the Thomas algorithm (tridiagonal Gaussian elimination).

3 Quadrature

Numerical quadrature aims to compute an approximation of a definite integral:

$$I(f) := \int_a^b f(x)\,dx$$

using a finite weighted sum of function evaluations:

$$Q(f) := \sum_{j=0}^{n} w_j f(c_j)$$

where $c_j \in [a, b]$ are the **quadrature nodes** and $w_j \in \mathbb{R}$ are the **quadrature weights**.

3.1 Polynomial Interpolation

Definition 3.1. Interpolatory Quadrature via Lagrange Basis: To approximate the integral of an unknown or complicated function $f(x)$, we interpolate f at $n+1$ distinct nodes $c_0, c_1, \ldots, c_n \in [a, b]$ with a polynomial $p_n \in \mathcal{P}_n$, and integrate p_n analytically:

$$I(f) \approx \int_a^b p_n(x)\,dx$$

Expressing the interpolant $p_n(x)$ in the **Lagrange basis** $L_j(x) = \prod_{k \neq j} \frac{x - c_k}{c_j - c_k}$:

$$p_n(x) = \sum_{j=0}^{n} f(c_j) L_j(x)$$

Integrating term-by-term yields:

$$\int_a^b p_n(x)\,dx = \sum_{j=0}^{n} \left(\int_a^b L_j(x)\,dx \right) f(c_j)$$

Thus, the quadrature weights are uniquely determined by integrating the Lagrange basis polynomials:

$$w_j = \int_a^b L_j(x)\,dx \quad \text{for } j = 0, 1, \ldots, n$$

Remark 3.2. An interpolatory quadrature rule with $n+1$ nodes has degree of exactness at least n: it integrates every polynomial $p \in \mathcal{P}_n$ exactly, because polynomial interpolation reproduces polynomials of degree $\leq n$ without error ($p_n \equiv p$).

Important 3.3. Runge's Phenomenon in Quadrature: Just as high-degree polynomial interpolation on **equidistant nodes** suffers from Runge's phenomenon (wild oscillations near the boundary), using equidistant nodes for high-order quadrature leads to severe instability. For $n \geq 8$, the weights $w_j = \int_a^b L_j(x)dx$ begin to

take on alternating negative signs, and:

$$\sum_{j=0}^{n} |w_j| \xrightarrow{n\to\infty} \infty \quad \text{exponentially}$$

Since the condition number of quadrature is $\sum |w_j|$, large opposing weights lead to **catastrophic numerical cancellation**. Therefore, high-degree polynomial interpolation on equidistant nodes should never be used for quadrature.

3.2 Composite Quadrature Rules

Definition 3.4. Domain Partitioning (h-Refinement): Instead of increasing the polynomial degree n over the whole interval, we keep the degree low and divide $[a, b]$ into N smaller subintervals of uniform width $h = \frac{b-a}{N}$:

$$a = x_0 < x_1 < x_2 < \cdots < x_N = b, \quad \text{where } x_i = a + ih$$

The total integral is the sum of integrals over each subinterval $[x_{i-1}, x_i]$:

$$\int_a^b f(x)\,dx = \sum_{i=1}^{N} \int_{x_{i-1}}^{x_i} f(x)\,dx$$

3.2.1 The Midpoint Rule

Definition 3.5. Midpoint Rule: Approximates each subinterval's integral by the area of a rectangle whose height is the function value at the subinterval midpoint $x_{i-1} + \frac{1}{2}h$:

$$Q_N^{MP}(f) := h \sum_{i=1}^{N} f\left(x_{i-1} + \frac{1}{2}h\right)$$

Theorem 3.6. Midpoint Error: For $f \in C^2[a, b]$, the composite midpoint error satisfies:

$$|I(f) - Q_N^{MP}(f)| = \frac{b-a}{24} h^2 |f''(\xi)| = \mathcal{O}(h^2) \quad \text{for some } \xi \in [a, b]$$

Remark 3.7. Intuition for the Midpoint "Bonus Degree": Although the midpoint rule approximates f with a degree-0 constant, it integrates linear functions $f(x) = c_1 x + c_0$ exactly! By symmetry around the midpoint m, the odd linear error term $(x - m)$ integrates to zero: the triangular error above the midpoint rectangle exactly cancels the triangular error below it.

3.2.2 The Trapezoidal Rule

Definition 3.8. Trapezoidal Rule: Approximates each subinterval's integral as the trapezoid spanned by its endpoints:

$$\begin{aligned} Q_N^{Trap}(f) &:= \sum_{i=1}^{N} \frac{h}{2}\Big(f(x_{i-1}) + f(x_i)\Big) \\ &= h\left[\frac{1}{2}f(x_0) + f(x_1) + f(x_2) + \cdots + f(x_{N-1}) + \frac{1}{2}f(x_N)\right] \end{aligned}$$

Every interior node is shared between adjacent intervals and receives weight h, while boundary endpoints receive weight $\frac{h}{2}$.

Theorem 3.9. Trapezoidal Error: For $f \in C^2[a,b]$, the composite trapezoidal error satisfies:

$$|I(f) - Q_N^{Trap}(f)| = \frac{b-a}{12}h^2|f''(\eta)| = \mathcal{O}(h^2) \quad \text{for some } \eta \in [a,b]$$

3.2.3 Simpson's Rule

Definition 3.10. Simpson's Rule: Interpolates f with a quadratic parabola over each subinterval using the two endpoints and the midpoint, weighted as $\frac{h}{6}(1,4,1)$:

$$Q_N^{Simp}(f) := \frac{h}{6}\sum_{i=1}^{N}\left[f(x_{i-1}) + 4f\left(\frac{x_{i-1}+x_i}{2}\right) + f(x_i)\right]$$

Theorem 3.11. Simpson's Error: Due to point symmetry around the midpoint, the cubic error term $(x-m)^3$ cancels out, making Simpson's rule exact for all cubic polynomials $\mathcal{P}_3$. For $f \in C^4[a,b]$:

$$|I(f) - Q_N^{Simp}(f)| = \frac{b-a}{2880}h^4|f^{(4)}(\xi)| = \mathcal{O}(h^4)$$

Halving the step size $h \to \frac{h}{2}$ reduces the error by a factor of $2^4 = 16$.

3.2.4 Smoothness

Remark 3.12. Non-Smooth Functions Degrade Convergence: The theoretical error bounds $\mathcal{O}(h^2)$ and $\mathcal{O}(h^4)$ strictly require bounded continuous derivatives ($f \in C^2$ or $f \in C^4$).

- If f has a **kink** (continuous, but first derivative jumps, e.g. $f(x) = |x-c|$), f'' does not exist as a bounded function. The convergence rate of both Trapezoidal and Simpson drops to $\mathcal{O}(h^1)$.
- If f has a **singularity** or infinite slope (e.g. $f(x) = \sqrt{x}$ near 0), the convergence

rate degrades further to $\mathcal{O}(h^{0.5})$.

High-order rules offer no advantage when the integrand lacks smoothness!

3.2.5 Romberg Extrapolation

Remark 3.13. textbfIntuition: By knowing the mathematical structure of the error, we can combine two relatively poor approximations to eliminate the dominant error term, creating a much higher-order method for free!.

Definition 3.14. Romberg Extrapolation for Quadrature: Applying Richardson extrapolation to the composite trapezoidal rule is called **Romberg Integration**. By the Euler-Maclaurin expansion, the trapezoidal error consists **exclusively of even powers of** h:

$$T(h) = I(f) + c_2h^2 + c_4h^4 + c_6h^6 + c_8h^8 + \dots$$

Note that intuitively this comes from the trapezoid being symetric hence $T(h) = T(-h)$, if there were odd terms this would not hold because of the changing sign. And because there are no odd powers (h^1, h^3, h^5), the leading error term has $p = 2$.

$$T_1(h/2) = \frac{4T(h/2) - T(h)}{4 - 1} = \frac{4T(h/2) - T(h)}{3}$$

This cancels the c_2h^2 error term, yielding an $\mathcal{O}(h^4)$ method. *Remarkable fact:* $T_1(h/2)$ is algebraically identical to **Simpson's rule**!

3.2.6 High-Order Rules

Remark 3.15. High-order **Newton-Cotes** rules fit a single high-degree polynomial over equidistant nodes on the entire interval. As established by Runge's phenomenon, this leads to negative weights for $n \geq 8$ and exponential divergence of condition numbers ($\sum |w_j| \to \infty$). Hence, high-degree equidistant rules are unstable and should be avoided.

Definition 3.16. Gauss Quadrature (Gauss-Legendre): Instead of fixing equidistant nodes, Gauss quadrature treats **both** the $n + 1$ nodes c_j and the $n + 1$ weights w_j as free variables ($2n + 2$ parameters).

- **Maximal Exactness:** Achieves the maximum theoretically possible degree of exactness:
$$d = 2n + 1 \quad \iff \quad \text{Order } q = 2n + 2$$
- **Orthogonality:** The nodes $c_0, \dots, c_n$ must be the $n+1$ roots of the $(n+1)$-th **Legendre polynomial** $P_{n+1}(x)$ on $[-1, 1]$.
- **Positivity & Stability:** All weights are strictly positive ($w_j > 0$), guarantee-

ing unconditional stability ($\sum |w_j| = b - a$).

- **Drawback:** Gauss nodes are **non-nested**: the nodes for n and $n+1$ do not overlap, so changing order requires throwing away all previous function evaluations.

Definition 3.17. Clenshaw-Curtis Quadrature: Clenshaw-Curtis quadrature interpolates f at the **Chebyshev-Gauss-Lobatto nodes** (extrema of Chebyshev polynomials, which include the endpoints):

$$x_j = \cos\left(\frac{j\pi}{n}\right) \quad \text{for } j = 0, 1, \ldots, n$$

and integrates the Chebyshev polynomial interpolant analytically.

Remark 3.18. Key Advantages of Clenshaw-Curtis:

1. **Suppresses Runge's Phenomenon:** Because Chebyshev nodes cluster near the boundaries with density $\propto \frac{1}{\sqrt{1-x^2}}$, the polynomial interpolation error is well-behaved and weights remain strictly positive for all practical n.
2. **Nested Nodes:** When doubling the number of intervals $n \to 2n$, all previous nodes are retained:

   $$x_j^{(n)} = \cos\left(\frac{j\pi}{n}\right) = \cos\left(\frac{2j\pi}{2n}\right) = x_{2j}^{(2n)}$$

   This allows adaptive doubling without wasting any prior function evaluations!
3. **Fast Computation via FFT:** The quadrature weights and evaluations can be computed in only $\mathcal{O}(n \log n)$ operations using the Fast Cosine Transform (FFT).
4. **Near-Gaussian Performance:** In practice, for smooth functions, Clenshaw-Curtis converges almost as quickly as Gauss quadrature while offering nested convenience.

3.3 Adaptive Quadrature

Remark 3.19. Motivation for Adaptivity: When an integrand f has a localized sharp spike, steep wave, or near-singularity, a uniform step size h is severely wasteful. To resolve the spike, h must be tiny everywhere, wasting computation in smooth regions. **Adaptive quadrature** refines h only where needed.

Definition 3.20. Local Error Estimation via Step Halving: On any subinterval $[a, b]$ with midpoint $m = \frac{a+b}{2}$:

- Compute a coarse estimate $I_{\text{coarse}} = Q(a, b)$ (e.g., single Simpson step of width $h = b - a$).
- Compute a fine estimate $I_{\text{fine}} = Q(a, m) + Q(m, b)$ (two Simpson steps of width $h/2$).
- Estimate the local truncation error:

$$\text{Est}(a, b) := \frac{1}{15}|I_{\text{fine}} - I_{\text{coarse}}| \approx |I(f) - I_{\text{fine}}|$$

(where the factor $\frac{1}{15} = \frac{1}{2^4-1}$ comes directly from Richardson extrapolation for Simpson's rule!).

Definition 3.21. Recursive Adaptive Algorithm: Given a user-specified global tolerance $\text{tol} > 0$:

1. If $\text{Est}(a, b) \leq \text{tol} \cdot \frac{b-a}{B-A}$ (where $[B, A]$ is the global domain): The local error is acceptable; return I_{fine}.
2. If $\text{Est}(a, b) > \text{tol} \cdot \frac{b-a}{B-A}$: Subdivide $[a, b]$ into $[a, m]$ and $[m, b]$, and recursively apply the algorithm to each half with local tolerance $\text{tol}/2$.

Remark 3.22. Intuition: Adaptive quadrature acts like a dynamic magnifying glass: it recursively zooms in with dense evaluations around sharp spikes and steep gradients, while taking large, efficient steps across flat, smooth regions.

3.4 Higher-Dimensional Quadrature

In scientific computing, we often need to integrate functions of multiple variables over a domain $\Omega \subset \mathbb{R}^d$:

$$I(f) := \int_\Omega f(\mathbf{x})\, d\mathbf{x} = \int \cdots \int f(x_1, \ldots, x_d)\, dx_1 \ldots dx_d$$

3.4.1 Tensor Product Rules (Standard Approach)

Theorem 3.23. For hyper-rectangular domains (e.g. $[a, b] \times [c, d]$ in 2D), the standard multidimensional quadrature is formed by taking the cartesian product of 1D quadrature rules. If dimension x uses 1D nodes and weights $(x_i, w_i^x)_{i=0}^{n_1-1}$ and dimension y uses $(y_j, w_j^y)_{j=0}^{n_2-1}$, the 2D tensor product rule is:

$$Q_{2D}(f) := \sum_{i=0}^{n_1-1} \sum_{j=0}^{n_2-1} w_i^x w_j^y\, f(x_i, y_j)$$

Remark 3.24. In d dimensions, each composite node is a tuple $(x_{i_1}^{(1)}, \ldots, x_{i_d}^{(d)})$ and its weight is the product of the corresponding 1D weights: $w_{\mathbf{i}} = \prod_{k=1}^{d} w_{i_k}^{(k)}$.

Code Example 3.25

```
import numpy as np

def full_grid_trapezoidal_2d(f, a, b, c, d, n1, n2):
    # 1D points and weights for x
    x = np.linspace(a, b, n1)
    wx = np.full(n1, (b - a) / (n1 - 1))
    wx[0] /= 2; wx[-1] /= 2

    # 1D points and weights for y
    y = np.linspace(c, d, n2)
    wy = np.full(n2, (d - c) / (n2 - 1))
    wy[0] /= 2; wy[-1] /= 2

    # Evaluate on the 2D tensor grid
    total = 0.0
    for i in range(n1):
        for j in range(n2):
            total += wx[i] * wy[j] * f(x[i], y[j])

    return total
```

Important 3.26. The Curse of Dimensionality: Suppose we use n quadrature points in each direction.

- **Exponential Point Explosion:** The total number of evaluations is $N = n^d$. For just $n = 10$ points per dimension in $d = 8$, $N = 10^8$ evaluations; in $d = 12$, $N = 10^{12}$ evaluations (prohibitively slow).
- **Deteriorating Convergence Rate:** If a 1D rule has error $\mathcal{O}(h^p) = \mathcal{O}(n^{-p})$, then expressing the error in terms of total function evaluations $N = n^d$:

$$\text{Error} = \mathcal{O}\left(N^{-p/d}\right)$$

  As dimension d grows, the convergence exponent $-p/d \to 0$. Halving the error in higher dimensions requires an astronomical increase in evaluations.

Consequently, full tensor grids become practically unusable for dimensions $d \geq 4$ or 5.

3.4.2 Sparse Grids (Smolyak / Combination Technique)

Remark 3.27. Full tensor grids waste almost all function evaluations on fine-scale resolutions in all directions simultaneously. For smooth functions whose mixed derivatives are bounded ($\left|\frac{\partial^{k_1+\cdots+k_d} f}{\partial x_1^{k_1} \ldots \partial x_d^{k_d}}\right| < \infty$), high-order cross-terms contribute very

little to the integral.

Definition 3.28. Sparse grids exploit this by combining anisotropic full grids that are fine in some directions and coarse in others, completely omitting grids that are simultaneously fine in all directions.

Theorem 3.29. For a target refinement level l, instead of evaluating one huge full grid of size $2^l \times 2^l$, we evaluate a series of much smaller rectangular grids and combine them:

$$Q_l^{\text{sparse}}(f) = \sum_{k_x+k_y=l+1} Q_{(k_x,k_y)}(f) - \sum_{k_x+k_y=l} Q_{(k_x,k_y)}(f)$$

Where the first sum adds up grids with complementary resolutions, while the second sum subtracts the redundant lower-level grids to cancel duplicate contributions.

Code Example 3.30

```
def sparse_grid_2d(f, level, a=0.0, b=1.0, c=0.0, d=1.0):
    """
    Computes a 2D Sparse Grid integral for target level l
    using the Combination Technique with composite trapezoidal rule
        .
    """
    total = 0.0
    l = level

    # 1. ADD terms where k_x + k_y = l + 1
    for k_x in range(1, l + 1):
        k_y = (l + 1) - k_x
        nx, ny = 2**k_x + 1, 2**k_y + 1
        total += full_grid_trapezoidal_2d(f, a, b, c, d, nx, ny)

    # 2. SUBTRACT terms where k_x + k_y = l (removes overlaps)
    for k_x in range(1, l):
        k_y = l - k_x
        nx, ny = 2**k_x + 1, 2**k_y + 1
        total -= full_grid_trapezoidal_2d(f, a, b, c, d, nx, ny)

    return total
```

3.4.3 Monte Carlo Integration

Remark 3.31. When dimension d is very large ($d \geq 10$) or the integration domain Ω is irregularly shaped, deterministic grid methods fail completely.

Definition 3.32. Monte Carlo integration treats the integral as the expected value of a random variable:

$$I(f) = \int_\Omega f(\mathbf{x})\, d\mathbf{x} = \mathrm{Vol}(\Omega) \cdot \mathbb{E}[f(\mathbf{X})], \quad \text{where } \mathbf{X} \sim \mathcal{U}(\Omega)$$

Sampling N independent and uniformly distributed random points $\mathbf{X}_1, \ldots, \mathbf{X}_N \overset{\text{i.i.d.}}{\sim} \mathcal{U}(\Omega)$, the integral is approximated by the sample average:

$$Q_N^{\mathrm{MC}}(f) := \frac{\mathrm{Vol}(\Omega)}{N} \sum_{i=1}^{N} f(\mathbf{X}_i)$$

Remark 3.33. By the Central Limit Theorem, the statistical error of Monte Carlo integration behaves as:

$$|I(f) - Q_N^{\mathrm{MC}}(f)| \approx \frac{\mathrm{Vol}(\Omega)\, \sigma(f)}{\sqrt{N}} = \mathcal{O}\left(N^{-1/2}\right)$$

where $\sigma(f)$ is the standard deviation of f on Ω.

Remark 3.34. Key Takeaways for Monte Carlo:

- **Immune to Dimension:** The convergence rate $\mathcal{O}(N^{-1/2})$ is **completely independent of the dimension** d! It is the same whether $d = 1$ or $d = 1000$.
- **Domain Flexibility:** Handles complex, non-rectangular geometries effortlessly (points outside the domain simply evaluate to 0).
- **Slow Convergence:** Gaining one decimal digit of precision requires $10^2 = 100$ times more sample points.

Code Example 3.35

```
def monte_carlo_nd(f, bounds, n_samples):
    """
    f: function f(x) where x is a 1D numpy array of length d
    bounds: list of (min, max) tuples for each dimension
    """
    bounds = np.array(bounds)
    d = len(bounds)
    volume = np.prod(bounds[:, 1] - bounds[:, 0])

    # Generate N uniform random samples in [0, 1)^d, scale to
        bounds
    samples = bounds[:, 0] + np.random.rand(n_samples, d) * (bounds
        [:, 1] - bounds[:, 0])

    # Evaluate and compute volume * mean(f)
    values = np.array([f(pt) for pt in samples])
```

```
    return volume * np.mean(values)
```

3.4.4 Quasi-Monte Carlo and Halton Sequences

Remark 3.36. The Clustering Problem of Pseudo-Random Points: Uniform random numbers generated by pseudo-random generators naturally suffer from random clustering: some regions get crowded with points, while other regions are left completely empty. This clumping is the sole reason for the slow $\mathcal{O}(N^{-1/2})$ convergence.

Definition 3.37. Quasi-Monte Carlo (QMC): Replaces random points with **deterministic, maximally spread-out** points called **low-discrepancy sequences**. These points deliberately avoid each other and fill space uniformly without forming regular grids.

Remark 3.38. The QMC error satisfies:

$$\text{Error} = \mathcal{O}\left(\frac{(\log N)^d}{N}\right) \approx \mathcal{O}(N^{-1})$$

which converges significantly faster than standard Monte Carlo ($\mathcal{O}(N^{-0.5})$) for moderate to high dimensions ($d \leq 15$).